

Laboratory 14:

Laplace Properties & System Functions Black Box System Identification: Reverse Engineering Unknown Hardware

From Measurements to Transfer Functions: Building a Digital Twin

Lab Duration: 2 hours

Pre-Lab Time Expected: 40 minutes (Laplace domain analysis and pole-zero theory)

Key Concept: You will be given a sealed "black box" circuit with unknown internal structure. Using only input/output measurements and Laplace domain system identification, you must mathematically reverse engineer its transfer function $H(s)$ and build a Simulink Digital Twin that perfectly replicates its behavior.

Contents

1	Real-World Context: System Identification Across Legacy and Modern Systems	4
1.1	Program 1: Aircraft Autopilot Modernization (Aerospace)	4
1.2	Program 2: Industrial Legacy Equipment Modernization (Manufacturing)	4
1.3	Program 3: Clinical Equipment Analysis (Biomedical)	5
1.4	The Unifying Methodology: Rigorous System Identification	5
2	Learning Objectives	6
3	Core Concepts: Laplace Domain and System Identification	7
3.1	Transfer Functions in the Laplace Domain	7
3.2	Poles and Zeros: The Foundation of Stability and Response	7
3.3	Step Response and Impulse Response	8
3.4	System Identification Methodology	9
4	Required Equipment and Software	10
5	Safety and Critical Wiring Notes	11
6	Hardware Setup: Testing the Black Box	12
6.1	Signal Generation and Data Acquisition Pipeline	12
6.2	Arduino PWM to Analog Conversion	12
6.3	Black Box Wiring	13
7	Pre-Lab Activity: Laplace Domain Analysis	14
7.1	Task 1: Poles and Settling Time	14
7.2	Task 2: Second-Order System Response	14
7.3	Task 3: Predict the Unknown Black Box Response	15
8	Measurement & Data Acquisition: Testing the Black Box	16
8.1	Task 1: Upload and Configure Test Signal Sketch	16

8.2	Task 2: Acquire Step Response Data	16
8.3	Task 3: Acquire Impulse Response Data	17
8.4	Task 4: Visual Assessment of Black Box Dynamics	17
9	Software Implementation: System Identification and Digital Twin	19
9.1	Task 1: Estimate the Transfer Function Using MATLAB	19
9.2	Task 2: Extract Poles, Zeros, and Gain	20
9.3	Task 3: Build the Simulink Digital Twin	20
9.4	Task 4: Validate the Digital Twin	21
9.5	Task 5: Frequency Response Validation (Optional Advanced)	22
10	Deliverables & TA Sign-Off: The Defense	23
A	Laplace Transform Table and Properties	26
A.1	Common Laplace Transform Pairs	26
A.2	Key Laplace Properties	26
B	First-Order and Second-Order System Characteristics	26
B.1	First-Order System	26
B.2	Second-Order System (Under-Damped)	26
C	System Identification Techniques	27
C.1	Logarithmic Decrement (For Oscillatory Responses)	27
C.2	Time-Constant Extraction (For First-Order Systems)	27
C.3	Least-Squares Fitting	27
D	MATLAB Code Templates	27
D.1	Complete System ID Workflow	27
E	Troubleshooting and Common Pitfalls	29
E.1	Data Acquisition Issues	29
E.2	System Identification Issues	29
E.3	Digital Twin Validation Issues	29
	References and Inspiration	30

Grading and Authenticity Requirements

Deliverable (1/2): This is a graded laboratory assignment. Include your **name, student ID, date, and unique black box identifier** on all handwritten derivations, MATLAB plots, and Simulink models.

Deliverable (2/2): Your hand-derived Laplace expressions and pole-zero analysis must be shown to the TA before hardware testing. Calculator results without derivations are not acceptable.

Checkpoint (1/10): Before final submission, verify that all MATLAB plots overlay the hardware data (raw measurements) with the Simulink digital twin output with clear legends distinguishing them. Include proper axis labels with units and your name/ID in the title.

1 Real-World Context: System Identification Across Legacy and Modern Systems

You are a Junior Systems Identification Engineer at a company specializing in reverse-engineering and modernizing legacy hardware systems. Your expertise in Laplace domain analysis and digital twins is in high demand across three critical industries: aerospace, industrial manufacturing, and biomedical devices. Each domain faces the same challenge: inherited systems with no documentation, no source code, and no original manufacturers—yet they must continue operating reliably.

1.1 Program 1: Aircraft Autopilot Modernization (Aerospace)

Your company modernizes vintage commercial and military aircraft from the 1960s–1990s. These planes have 20+ years of remaining service life but their analog autopilot controllers are manufactured by companies that have long since closed or been acquired.

The challenge: Aircraft autopilots are safety-critical systems. A wrong control law update causes loss of aircraft. Before engineers can design a modern digital replacement, they must understand exactly what the old analog hardware is doing—its transfer function $H(s)$.

Your task: You are given a sealed autopilot pitch-damper module (the component that smooths out turbulent pitch oscillations). Using step and impulse test signals, you must identify its transfer function from measured input/output data. Once verified, the digital twin enables: (1) simulation of failure modes safely, (2) design of a modern digital pitch damper that matches original behavior, (3) validation that the new system meets FAA requirements.

Regulatory requirement: Your identified transfer function must predict the system’s settling time to within $\pm 10\%$ and pole locations to within $\pm 5\%$ of the true values. Regulators demand this precision because autopilot instability can cause crashes.

1.2 Program 2: Industrial Legacy Equipment Modernization (Manufacturing)

Your company helps factories modernize production lines running legacy equipment: 1970s–1990s CNC machines, hydraulic systems, and process controllers manufactured by defunct companies.

The challenge: A factory with a 20-year-old injection molding machine needs predictive maintenance and process optimization. The hydraulic pressure control system is a sealed black box—repairmen know how to tune it through trial-and-error, but no one understands the underlying dynamics. Any modification risks production downtime and costly rework.

Your task: You identify the hydraulic servo system’s transfer function by injecting pressure step commands and recording the response. The identified model reveals: (1) whether the system is damped or oscillatory, (2) settling time and overshoot, (3) bandwidth limitations. Armed with this model, engineers can redesign the control algorithm to improve cycle time (e.g., 5–10% faster production) and reduce quality variability (e.g., tighter part tolerances).

Business requirement: A 5% production speed improvement is worth millions in annual throughput. Your system ID model must be accurate enough to support control redesign that is production-certified without physical hardware trials (too expensive and risky).

1.3 Program 3: Clinical Equipment Analysis (Biomedical)

Your company supports hospitals deploying vintage anesthesia delivery systems, ventilators, and patient monitors. Regulatory changes require that all equipment undergo risk assessments and functional safety analyses. But manufacturers are long gone.

The challenge: An anesthesia machine from 1985 uses an analog servo to control vaporizer output (mixing anesthetic gas with oxygen). The servo valve's response characteristics are unknown. Incorrect servomechanism behavior can lead to patient overdose (dangerous) or underdose (surgery awareness, also dangerous). The hospital cannot safely modify the equipment without understanding its dynamics.

Your task: You identify the servo valve transfer function by injecting small step changes in commanded flow and measuring actual flow response. The transfer function reveals: (1) latency (is there significant delay?), (2) overshoot risk (does the servo "ring" and overshoot setpoint?), (3) settling time (how long does adjustment take?). This information feeds into a functional safety analysis (FMEA) to assess risk and determine if the equipment is still safe to operate.

Clinical and regulatory requirement: Your analysis must achieve $> 95\%$ prediction accuracy for doses in the therapeutic range. Even 1% error in predicting vaporizer response could mean the difference between safe anesthesia and patient harm. The FDA requires documentation of this validation before the equipment continues clinical use.

1.4 The Unifying Methodology: Rigorous System Identification

All three programs depend on the same engineering discipline:

1. **Test signal design:** Step inputs reveal transient response (damping, settling time). Impulse inputs reveal raw system dynamics. Chirp (swept sine) inputs map frequency response.
2. **Data acquisition:** Careful measurement with adequate sampling rate ($\geq 10\times$ system bandwidth) and noise filtering to extract true system response.
3. **Transfer function estimation:** Use MATLAB's `tfest()` or manual pole-zero fitting to estimate $H(s)$ from the measured data. Conservatism is critical: a simple model (fewer poles/zeros) that is robust is better than a complex model that overfits to noise.
4. **Validation and quantification of error:** Build a Simulink digital twin with the identified $H(s)$ and overlay its output against all measured data. Quantify error metrics: peak overshoot error, settling time error, steady-state gain error, etc.
5. **Understand limitations:** Your model captures linear, time-invariant behavior. Real systems have nonlinearities, time-varying parameters, and unmeasured disturbances. Good engineers document these limitations and understand when their model breaks down.

In this lab, you will practice this complete workflow on a mystery black-box system, building a transfer function model and defending it against measurement data—preparing you for real-world system identification in aerospace, manufacturing, and biomedical applications.

2 Learning Objectives

By the end of this lab, you should be able to:

1. **Understand Laplace transform properties:** Recognize how linearity, scaling, differentiation, and integration properties allow engineers to predict system behavior from the transfer function.
2. **Relate poles and zeros to system response:** Explain how pole locations in the s -plane dictate natural frequency, damping ratio, settling time, and stability.
3. **Derive step and impulse responses:** Hand-derive the time-domain output for a given $H(s)$ when excited by step or impulse inputs.
4. **Design system identification test signals:** Understand why step, impulse, and swept-sine inputs are chosen for system ID (each reveals different aspects of $H(s)$).
5. **Acquire clean experimental data:** Use the Arduino and MATLAB to record input/output data from the black box with appropriate sampling rates and noise filtering.
6. **Identify transfer functions from data:** Use MATLAB's `tfest()` function or manual curve-fitting to estimate $H(s)$ from measured responses.
7. **Build and validate digital twins:** Create a Simulink transfer function model and overlay its output against hardware measurements to assess fidelity.
8. **Defend your identification:** Articulate the strengths and limitations of your identified model, quantify error metrics, and explain discrepancies between simulation and hardware.

3 Core Concepts: Laplace Domain and System Identification

3.1 Transfer Functions in the Laplace Domain

For a linear, time-invariant (LTI) system, the **transfer function** $H(s)$ is defined as the ratio of the Laplace transform of the output to the Laplace transform of the input:

$$H(s) = \frac{Y(s)}{X(s)}, \quad (1)$$

where:

- $X(s) = \mathcal{L}\{x(t)\}$ is the Laplace transform of the input signal $x(t)$.
- $Y(s) = \mathcal{L}\{y(t)\}$ is the Laplace transform of the output signal $y(t)$.
- $s = \sigma + j\omega$ is the complex Laplace variable.

The transfer function completely characterizes the system's behavior for any input, as long as the system is linear and time-invariant.

3.2 Poles and Zeros: The Foundation of Stability and Response

The transfer function is usually written as a ratio of polynomials:

$$H(s) = K \frac{(s - z_1)(s - z_2) \cdots (s - z_m)}{(s - p_1)(s - p_2) \cdots (s - p_n)}, \quad (2)$$

where:

- **Zeros** ($z_1, z_2, \dots, z_m$) are the values of s that make the numerator zero.
- **Poles** ($p_1, p_2, \dots, p_n$) are the values of s that make the denominator zero.
- **Gain** K is the steady-state scaling factor.

Key insight: The locations of poles in the complex s -plane determine the system's response characteristics:

- **Real pole at** $s = -\alpha$: Contributes exponential decay $e^{-\alpha t}$ to the response. Smaller $|\alpha|$ means slower decay (longer time constant $\tau = 1/\alpha$).
- **Complex poles at** $s = -\alpha \pm j\omega_d$: Contribute oscillatory exponential decay $e^{-\alpha t} \cos(\omega_d t + \phi)$. The real part α controls decay rate; the imaginary part ω_d is the damped oscillation frequency.
- **Poles on the right half-plane** (real part > 0): System is **unstable**—exponential growth rather than decay.
- **Poles on the imaginary axis**: System is marginally stable—oscillations at constant amplitude.

For a second-order system with complex poles, it is convenient to express the pole locations in terms of the **natural frequency** ω_n and **damping ratio** ζ :

$$p_{1,2} = -\zeta\omega_n \pm j\omega_n\sqrt{1 - \zeta^2}. \quad (3)$$

The damping ratio determines the character of the response:

- $\zeta > 1$: Over-damped (no overshoot, slow settling).
- $\zeta = 1$: Critically damped (fastest settling without overshoot).
- $0 < \zeta < 1$: Under-damped (overshoot, oscillation, faster settling than critical).
- $\zeta = 0$: Marginally stable (pure oscillation at ω_n).

[INSERT DIAGRAM: Pole-Zero Plot showing real poles, complex pole pairs, and their relationship to time-domain response (exponential decay, oscillation, etc.)]

3.3 Step Response and Impulse Response

Step Response

For a step input $x(t) = A \cdot u(t)$ (Laplace transform $X(s) = A/s$), the output is:

$$Y(s) = H(s) \cdot \frac{A}{s}. \quad (4)$$

The **step response** $y_{\text{step}}(t) = \mathcal{L}^{-1}\{Y(s)\}$ shows how the system reaches steady state. Key metrics:

- **Steady-state value:** $y_{\infty} = A \cdot H(0) = A \cdot K$ (the DC gain).
- **Settling time:** How long until $|y(t) - y_{\infty}| < \epsilon$ for some tolerance ϵ .
- **Overshoot:** The percentage by which the peak exceeds steady state.
- **Rise time:** Time for output to go from 10% to 90% of final value.

Impulse Response

For an impulse input $x(t) = \delta(t)$ (Laplace transform $X(s) = 1$), the output is:

$$Y(s) = H(s) \cdot 1 = H(s). \quad (5)$$

The **impulse response** $h(t) = \mathcal{L}^{-1}\{H(s)\}$ is the system's response to a brief, intense shock. The impulse response directly encodes all the dynamic characteristics of the system and can be used to compute the response to any input via convolution:

$$y(t) = \int_0^t h(\tau) x(t - \tau) d\tau. \quad (6)$$

3.4 System Identification Methodology

System identification is the process of estimating $H(s)$ (or an equivalent discrete-time model) from measured input/output data. The general approach is:

1. **Design test signals:** Choose inputs that will excite the system dynamics well. Step and impulse are common because their Laplace transforms are simple.
2. **Acquire data:** Record $y(t)$ while applying $x(t)$ to the system. Typically, record at a high sampling rate ($10\text{--}100\times$ the system's bandwidth) and average over multiple trials to reduce noise.
3. **Estimate the model:** Use curve-fitting or formal identification algorithms to fit $H(s)$ to the data. Common methods include:
 - **Logarithmic decrement:** Measure the decay envelope of an oscillatory response to extract damping ratio.
 - **Time-constant extraction:** For first-order systems, measure when the step response reaches 63.2% of steady state.
 - **Least-squares fitting:** Use MATLAB's `tfest()` to solve an optimization problem that finds the best-fitting $H(s)$.
4. **Validate the model:** Compare the predicted output of your identified $H(s)$ (in Simulink) to the actual hardware output. If they match closely, your identification is successful.
5. **Quantify uncertainty:** Compute error metrics (RMS error, percent overshoot difference, settling time difference) and state your confidence in the model.

[INSERT DIAGRAM: System Identification Flow (block diagram showing: Unknown System $\rightarrow$ Test Signals $\rightarrow$ Measure Response $\rightarrow$ Curve Fit to find $H(s)$ $\rightarrow$ Validate with Simulink)]

4 Required Equipment and Software

Table 1. Required Hardware and Software

Category	Required Items
Black box system	TA-provided sealed circuit (unique to each pair, unknown transfer function)
Signal generation	Arduino Uno with USB cable (running skeleton test signal code)
DAC or PWM output	Arduino PWM pin (Pin 9 or Pin 11) driving a low-pass filter to create analog signal
ADC input	Arduino ADC pin (A0 or A1) reading the black box output
Measurement device	Multimeter (to verify DC levels) and oscilloscope (optional, for visual verification)
Prototyping	Breadboard, jumper wires, resistors (1 k Ω , 10 k Ω), capacitors (0.1 μ F, 1 μ F)
Software	MATLAB R2021a or later with System Identification Toolbox
Software	Simulink with Control System Toolbox
Provided	<code>SysID_TestSignals_Skeleton.ino</code> (Arduino sketch for test signal generation)
Provided	<code>SysID_DataLogger.m</code> (MATLAB script for data acquisition)
Provided	<code>SysID_Blank_Simulink.slx</code> (blank Simulink model template)

Black Box Specifications (TA-Provided)

Each black box will have:

- **Input range:** 0–5 V (compatible with Arduino DAC/PWM output).
- **Output range:** 0–5 V (compatible with Arduino ADC input).
- **Bandwidth:** Approximately 1 kHz to 100 kHz (typical for analog circuits).
- **Unique transfer function:** Every pair has a different $H(s)$. You cannot share answers.
- **Sealed case:** The circuit is physically sealed; you cannot open it or probe internal nodes. All identification must be done via external measurements.

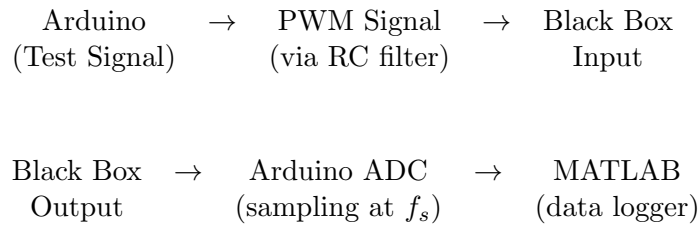
5 Safety and Critical Wiring Notes

- **Black Box Power:** The TA will power the black box. Do NOT attempt to power it yourself—you may damage it or receive electric shock.
- **Input Signal Level:** Keep the test signal between 0–5 V. Exceeding this range will damage the Arduino ADC or the black box input.
- **Common Ground:** The Arduino, black box, and all test equipment must share a common ground. Floating grounds cause measurement errors and noise.
- **Oscilloscope Probe Grounding:** If using an oscilloscope, connect the probe ground to the circuit ground to avoid noise coupling and ground loops.
- **Do Not Touch the Black Box:** It is sealed and temperature-sensitive. Avoid touching it with bare hands; use gloves if necessary.

6 Hardware Setup: Testing the Black Box

6.1 Signal Generation and Data Acquisition Pipeline

The test flow is:



6.2 Arduino PWM to Analog Conversion

The Arduino's PWM (Pulse-Width Modulation) output at Pin 9 or Pin 11 is a digital square wave. To convert it to an analog test signal, a simple RC low-pass filter smooths the PWM:

Student Notes: RC Filter Configuration:

Resistor: $R = 10 \text{ k}\Omega$

Capacitor: $C = 0.1 \text{ }\mu\text{F}$

Cutoff frequency: $f_c = 1/(2\pi RC) \approx 159 \text{ Hz}$

This ensures the PWM carrier (490 Hz native frequency on most Arduinos) is filtered out, leaving only the low-frequency test signal.

[INSERT SCHEMATIC: Arduino PWM → RC Filter → Black Box Input (showing pin labels and component values)]

6.3 Black Box Wiring

Student Notes: Black Box Connections:

Signal Input $\leftarrow$ RC filter output

Signal Output $\rightarrow$ Arduino A0 (ADC input)

GND (black box) $\leftrightarrow$ Arduino GND (shared reference)

Power (black box) $\leftarrow$ TA-supplied (do not modify)

[INSERT SCHEMATIC: Complete Test Setup (Arduino, RC filter, Black Box, ADC input, showing all pin labels and ground connections)]

7 Pre-Lab Activity: Laplace Domain Analysis

7.1 Task 1: Poles and Settling Time

Consider a generic first-order system with a pole at $s = -\alpha$:

$$H(s) = \frac{K}{s + \alpha}. \quad (7)$$

Question: What is the relationship between the pole location α and the settling time of the system?

Pre-lab Deliverable (1/4): Derive the step response $y_{\text{step}}(t)$ for this first-order system (assume step input $x(t) = u(t)$, so $X(s) = 1/s$). Show all algebraic steps and identify the time constant $\tau = 1/\alpha$.

Student Notes: First-Order Step Response Derivation:

7.2 Task 2: Second-Order System Response

Consider a second-order system with complex poles at $s = -\zeta\omega_n \pm j\omega_n\sqrt{1 - \zeta^2}$ (where $0 < \zeta < 1$):

$$H(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}. \quad (8)$$

Pre-lab Deliverable (2/4): Explain (qualitatively, no need to fully derive) how the damping ratio ζ affects the overshoot and settling time of the step response. What value of ζ gives the fastest settling with zero overshoot?

Student Notes: Second-Order System Analysis:**7.3 Task 3: Predict the Unknown Black Box Response**

You do not know the internal structure of your black box, but you can predict general behaviors. Suppose your black box's transfer function is either:

- **Option A:** A first-order low-pass filter, $H(s) = \frac{\omega_c}{s + \omega_c}$ with $\omega_c = 2\pi \times 1000$ rad/s (i.e., $f_c = 1$ kHz).
- **Option B:** A second-order under-damped system with $\omega_n = 2\pi \times 5000$ rad/s and $\zeta = 0.5$.

Pre-lab Deliverable (3/4): For each option, predict the approximate step response: steady-state value, settling time (to 5% tolerance), and whether overshoot occurs.

Student Notes: Predicted Black Box Response:

8 Measurement & Data Acquisition: Testing the Black Box

8.1 Task 1: Upload and Configure Test Signal Sketch

1. Load the provided `SysID_TestSignals_Skeleton.ino` into the Arduino IDE.
2. This sketch will generate three test signals on Pin 9 (PWM output):
 - (a) **Step input:** PWM value jumps from 0 to 255 and holds.
 - (b) **Impulse approximation:** Very narrow pulse (1–10 ms duration).
 - (c) **Chirp (swept sine):** Frequency increases from 10 Hz to 1 kHz over 5 seconds.
3. Verify that the sketch compiles without errors.

Checkpoint (2/10): Show the TA that the Arduino is successfully running the test sketch. Confirm the Pin 9 PWM output is toggling (visible on an oscilloscope or multimeter reading the RC filter output).

8.2 Task 2: Acquire Step Response Data

1. Run the MATLAB script `SysID_DataLogger.m`. This script:
 - (a) Opens a serial port connection to the Arduino.
 - (b) Triggers the step input test signal.
 - (c) Records both the filtered PWM input (Pin 9) and the black box output (A0) for 5 seconds at 10 kHz sampling rate.
 - (d) Saves the data to `step_response.mat`.
2. Plot the recorded data (input and output) in MATLAB:

```
%% Load and plot step response
load('step_response.mat'); % Contains: t, u_step (input), y_step (output)

figure('Position', [100, 100, 1000, 600]);
subplot(2, 1, 1);
plot(t, u_step, 'b-', 'LineWidth', 1.5);
ylabel('Input□(V)');
title('Test□Signal:□Step□Input');
grid on;

subplot(2, 1, 2);
plot(t, y_step, 'r-', 'LineWidth', 1.5);
ylabel('Output□(V)');
xlabel('Time□(s)');
title('Black□Box□Response□to□Step□Input');
grid on;

sgtitle('Lab□14:□Step□Response□Data□Acquisition');
```

Checkpoint (3/10): Generate the plot showing your recorded step response. Visually inspect the output waveform and describe its characteristics: Does it overshoot? Is it oscillatory? What is the approximate settling time?

[INSERT MATLAB PLOT: Step Response Data (input and output waveforms)]

8.3 Task 3: Acquire Impulse Response Data

Repeat the data acquisition with the impulse test signal:

1. Modify the Arduino sketch to generate the impulse test (very narrow pulse).
2. Run the MATLAB data logger again, saving to `impulse_response.mat`.
3. Plot the input and output as above.

Checkpoint (4/10): Generate the plot showing your impulse response. The impulse response directly encodes the transfer function's characteristics. Describe what you observe.

[INSERT MATLAB PLOT: Impulse Response Data (input pulse and output waveform)]

8.4 Task 4: Visual Assessment of Black Box Dynamics

From your step and impulse data, answer the following questions:

Discussion (1/2): Black Box Characteristics:

Student Notes: 1. Is the black box system stable (does the output return to steady state)?

2. What is the approximate steady-state gain (ratio of final output to step input)?

3. Does the black box appear to be first-order, second-order, or higher?

4. Is there any observable overshoot or oscillation?

9 Software Implementation: System Identification and Digital Twin

9.1 Task 1: Estimate the Transfer Function Using MATLAB

Use MATLAB's System Identification Toolbox to fit a transfer function model to your measured data:

```
%% System Identification using tfest()
load('step_response.mat');

% Create a time series data object
sys_data = iddata(y_step, u_step, 1/Fs); % output, input, sample time

% Estimate a first-order transfer function
% nb = 1 (first-order numerator), na = 1 (first-order denominator)
sys_fo = tfest(sys_data, 1, 1);

% Also estimate a second-order system for comparison
sys_so = tfest(sys_data, 2, 2);

% Display the identified models
disp('Identified First-Order System:');
disp(sys_fo);

disp('Identified Second-Order System:');
disp(sys_so);

% Compare predictions with measured data
y_pred_fo = lsim(sys_fo, u_step, t);
y_pred_so = lsim(sys_so, u_step, t);

figure('Position', [100, 100, 1000, 600]);
plot(t, y_step, 'k-', 'LineWidth', 1.5, 'DisplayName', 'Hardware');
hold on;
plot(t, y_pred_fo, 'b--', 'LineWidth', 1.5, 'DisplayName', '1st-Order Model');
plot(t, y_pred_so, 'r--', 'LineWidth', 1.5, 'DisplayName', '2nd-Order Model');
xlabel('Time (s)');
ylabel('Output (V)');
title('System ID: Measured vs. Model Predictions');
legend('Location', 'best');
grid on;

% Compute error metrics
err_fo = y_step - y_pred_fo;
err_so = y_step - y_pred_so;
rmse_fo = sqrt(mean(err_fo.^2));
rmse_so = sqrt(mean(err_so.^2));
fprintf('RMSE (1st-order): %.4f V\n', rmse_fo);
fprintf('RMSE (2nd-order): %.4f V\n', rmse_so);
```

Checkpoint (5/10): Run the system identification code and compare the first-order and second-order models. Which provides a better fit to your hardware data? Record the identified transfer function with its specific pole and zero locations.

[INSERT MATLAB PLOT: Hardware Data vs. Fitted Models (showing measured step response overlaid with first and second-order predictions)]

9.2 Task 2: Extract Poles, Zeros, and Gain

From the identified system, explicitly write out the transfer function:

Pre-lab Deliverable (4/4): Your Identified Transfer Function:

Student Notes: Transfer Function $H(s)$:

$H(s) =$

Poles:

Zeros:

Gain (DC):

9.3 Task 3: Build the Simulink Digital Twin

1. Open the provided SysID.Blank.Simulink.slx template.
2. Drag a “Transfer Fcn” block into the model. Double-click it and enter your identified pole-zero-gain form.
3. Connect a “From Workspace” block (carrying your measured input u_{step}) to the Transfer Fcn input.
4. Connect the Transfer Fcn output to a “To Workspace” block to save the simulated output.
5. Run the simulation with the exact same input u_{step} you applied to the hardware.

[INSERT SIMULINK SCREENSHOT: Digital Twin Model (showing From Workspace, Transfer Fcn block with your $H(s)$, and To Workspace)]

9.4 Task 4: Validate the Digital Twin

Compare the Simulink output to your hardware measurements:

```
% Validation: Hardware vs. Simulink Digital Twin
load('step_response.mat');
load('simulink_output.mat'); % y_sim from Simulink

figure('Position', [100, 100, 1000, 600]);
plot(t, y_step, 'b-', 'LineWidth', 2, 'DisplayName', 'Hardware(BlackBox)');
hold on;
plot(t, y_sim, 'r--', 'LineWidth', 2, 'DisplayName', 'Simulink(DigitalTwin)');
xlabel('Time(s)');
ylabel('Output(V)');
title('BlackBoxSystemIdentification:Hardwarevs.DigitalTwin');
legend('Location', 'best');
grid on;

% Compute agreement metrics
err = y_step - y_sim;
rmse = sqrt(mean(err.^2));
max_err = max(abs(err));
fprintf('RMSE: %.4fV\n', rmse);
fprintf('MaxError: %.4fV\n', max_err);
fprintf('PercentError(RMS): %.2f%%\n', 100 * rmse / max(y_step));
```

Checkpoint (6/10): Generate the overlay plot showing hardware and Simulink output. Compute error metrics (RMSE, max error, percent error) and assess whether your Digital Twin is accurate enough (typically < 10% RMS error is considered good).

[INSERT MATLAB PLOT: Hardware vs. Digital Twin Overlay (showing how closely Simulink matches hardware)]

9.5 Task 5: Frequency Response Validation (Optional Advanced)

For additional confidence, run the chirp sweep test and compare frequency responses:

1. Acquire the chirp sweep response from the hardware (using the test sketch).
2. Compute the FFT of the chirp response to extract magnitude and phase vs. frequency.
3. Generate the same chirp signal through your Simulink Digital Twin and compute its FFT.
4. Overlay the two frequency responses (magnitude and phase).

Discussion (2/2): Does the frequency response of your identified system match the hardware? Where do they diverge? What could cause the discrepancies?

Student Notes:

10 Deliverables & TA Sign-Off: The Defense

Four Checkpoints Leading to the Final Defense

Your lab culminates in a verbal “Defense” to the TA, where you must justify your system identification methodology and present your Digital Twin. Work efficiently to finish all checkpoints within the 2-hour window.

Checkpoint 1: Pre-Lab Laplace Theory (Target: 15 min)

Checkpoint (7/10): Before hardware testing, show the TA:

- ☐ Hand-derived step response for a first-order system (Task 1). Show all algebraic steps.
- ☐ Qualitative explanation of how damping ratio ζ affects overshoot and settling time (Task 2).
- ☐ Predictions for your black box response under step input (Task 3): expected settling time, overshoot, and characteristic response.
- ☐ TA verifies: All mathematical derivations are correct and predictions are reasonable.

TA Signature: _____ **Time:** _____

Checkpoint 2: Data Acquisition and Measurement (Target: 25 min)

Checkpoint (8/10): Show the TA:

- ☐ Working Arduino test signal generation verified on Pin 9 (TA observes step, impulse, and chirp signals).
- ☐ Successfully acquired step and impulse response data from the black box using the MATLAB data logger.
- ☐ Plots of both the test input signal and black box output response (as shown in Tasks 2 and 3).
- ☐ Verbal description of the black box’s observed dynamics (stable/unstable, oscillatory/non-oscillatory, settling time estimate).
- ☐ TA verifies: Data quality is acceptable (minimal noise, clear signal) and matches your pre-lab predictions reasonably.

TA Signature: _____ **Time:** _____

Checkpoint 3: System Identification and Transfer Function (Target: 25 min)

Checkpoint (9/10): Show the TA:

- ☐ MATLAB results from `tfest()`: Display the identified first-order and second-order transfer functions.
- ☐ Plot overlaying the measured hardware step response with predictions from both models (Task 1).

- ☐ Quantitative error metrics: RMSE and percent error for each model fit.
- ☐ Your chosen model (first vs. second-order) with explicit statement of poles, zeros, and gain.
- ☐ TA verifies: The identified transfer function is reasonable (poles stable, gain sensible, error metrics acceptable).

TA Signature: _____ **Time:** _____

Checkpoint 4: Digital Twin Validation and Defense (Target: 25 min)

Checkpoint (10/10): Show the TA and be prepared to defend:

- ☐ Simulink Digital Twin model screenshot showing the Transfer Fcn block with your identified $H(s)$.
- ☐ Overlay plot: Hardware step response vs. Simulink simulation output (Task 4).
- ☐ Error metrics: RMSE, max error, and percent RMS error between hardware and simulation.
- ☐ Written explanation: How well does your Digital Twin replicate the hardware? Where do they diverge? Why?
- ☐ Verbal defense: Explain your system identification methodology, the choice of model order, and confidence level in your Digital Twin.

Defense Topics You Must Be Prepared to Discuss:

1. Why did you choose first-order vs. second-order? What evidence from your data supports this choice?
2. What are the limitations of your identified model? Are there unmodeled dynamics or nonlinearities?
3. How sensitive is your system ID to measurement noise? Did you filter the data?
4. If you tested with a different input (e.g., the chirp sweep), does the Digital Twin still match?
5. How confident are you in your model for predicting the system's behavior in unknown scenarios?

TA Signature: _____ **Time:** _____

Final Grade Summary

Checkpoint	Points	Signed Off
1. Pre-Lab Laplace Theory	20	☐
2. Data Acquisition and Measurement	25	☐
3. System Identification and Transfer Function	25	☐
4. Digital Twin Validation and Defense	30	☐
TOTAL	100	

Final TA Signature: _____ **Final Grade:** _____

Notes/Comments from TA:

Student Notes:

A Laplace Transform Table and Properties

A.1 Common Laplace Transform Pairs

Signal $x(t)$	Laplace Transform $X(s)$
$\delta(t)$ (impulse)	1
$u(t)$ (unit step)	$1/s$
$t \cdot u(t)$	$1/s^2$
$e^{-at}u(t)$	$1/(s+a)$
$\cos(\omega t)u(t)$	$s/(s^2 + \omega^2)$
$\sin(\omega t)u(t)$	$\omega/(s^2 + \omega^2)$
$e^{-at} \cos(\omega t)u(t)$	$(s+a)/[(s+a)^2 + \omega^2]$
$e^{-at} \sin(\omega t)u(t)$	$\omega/[(s+a)^2 + \omega^2]$

A.2 Key Laplace Properties

- **Linearity:** $\mathcal{L}\{ax_1(t) + bx_2(t)\} = aX_1(s) + bX_2(s)$.
- **Time shift:** $\mathcal{L}\{x(t-T)u(t-T)\} = e^{-sT}X(s)$.
- **Frequency shift:** $\mathcal{L}\{e^{-at}x(t)\} = X(s+a)$.
- **Differentiation:** $\mathcal{L}\{dx/dt\} = sX(s) - x(0^-)$.
- **Integration:** $\mathcal{L}\{\int_0^t x(\tau)d\tau\} = X(s)/s$.
- **Final Value Theorem:** $\lim_{t \rightarrow \infty} x(t) = \lim_{s \rightarrow 0} sX(s)$ (if the limit exists).
- **Initial Value Theorem:** $x(0^+) = \lim_{s \rightarrow \infty} sX(s)$.

B First-Order and Second-Order System Characteristics

B.1 First-Order System

Standard form: $H(s) = \frac{\omega_c}{s + \omega_c}$ or $H(s) = \frac{K}{\tau s + 1}$.

Step response: $y(t) = K(1 - e^{-t/\tau})u(t)$.

Key metrics:

- Time constant: $\tau = 1/\omega_c$.
- Settling time (to 5%): $t_s \approx 3\tau$.
- No overshoot.

B.2 Second-Order System (Under-Damped)

Standard form: $H(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2}$ (where $0 < \zeta < 1$).

Step response:

$$y(t) = 1 - e^{-\zeta\omega_n t} \left[\cos(\omega_d t) + \frac{\zeta}{\sqrt{1-\zeta^2}} \sin(\omega_d t) \right], \quad (9)$$

where $\omega_d = \omega_n \sqrt{1-\zeta^2}$ is the damped natural frequency.

Key metrics:

- Percent overshoot: $PO = 100 \cdot e^{-\zeta\pi/\sqrt{1-\zeta^2}}$.
- Settling time (to 2%): $t_s \approx 4/(\zeta\omega_n)$.
- Peak time: $t_p = \pi/\omega_d$.

C System Identification Techniques

C.1 Logarithmic Decrement (For Oscillatory Responses)

If the system's step response exhibits oscillation, the damping ratio can be extracted from the oscillation's decay:

$$\delta = \ln \left(\frac{y_1}{y_2} \right), \quad (10)$$

where y_1 and y_2 are successive peak amplitudes. Then:

$$\zeta = \frac{\delta}{\sqrt{4\pi^2 + \delta^2}}. \quad (11)$$

C.2 Time-Constant Extraction (For First-Order Systems)

For a first-order system, the step response reaches 63.2% of final value at time $t = \tau$. You can read τ directly from the measured step response and compute the pole as $p = -1/\tau$.

C.3 Least-Squares Fitting

MATLAB's `tfest()` solves a least-squares optimization problem:

$$\min_{\theta} \sum_{k=1}^N |y[k] - \hat{y}[k](\theta)|^2, \quad (12)$$

where θ represents the unknown model parameters (pole, zero, gain), $y[k]$ is the measured output, and $\hat{y}[k]$ is the predicted output. The algorithm converges to the best-fitting model of the chosen order.

D MATLAB Code Templates

D.1 Complete System ID Workflow

```
%% Lab 14: Complete System Identification Workflow
```

```
clear; clc;
```

```
%% Load experimental data
```

```
load('step_response.mat'); % Assuming Fs (sampling rate) is saved
```

```
% Create iddata object
```

```
sys_data = iddata(y_step, u_step, 1/Fs);
```

```
% Estimate transfer functions of increasing order
sys_1 = tfest(sys_data, 1, 1); % 1st order
sys_2 = tfest(sys_data, 2, 2); % 2nd order (might include zeros)

% Simulate predicted outputs
y_pred_1 = lsim(sys_1, u_step, t);
y_pred_2 = lsim(sys_2, u_step, t);

% Compare models
figure('Position', [100, 100, 1200, 700]);
subplot(2, 2, 1);
plot(t, y_step, 'k-', 'LineWidth', 1.5, 'DisplayName', 'Hardware');
hold on;
plot(t, y_pred_1, 'b--', 'LineWidth', 1.5, 'DisplayName', '1st-Order');
plot(t, y_pred_2, 'r--', 'LineWidth', 1.5, 'DisplayName', '2nd-Order');
xlabel('Time (s)');
ylabel('Output (V)');
title('Step Response Comparison');
legend;
grid on;

% Error analysis
err_1 = y_step - y_pred_1;
err_2 = y_step - y_pred_2;
rmse_1 = sqrt(mean(err_1.^2));
rmse_2 = sqrt(mean(err_2.^2));

subplot(2, 2, 2);
semilogy(t, abs(err_1), 'b-', 'LineWidth', 1.5, 'DisplayName', ['1st-Order RMSE=' num2str(rmse_1)]);
hold on;
semilogy(t, abs(err_2), 'r-', 'LineWidth', 1.5, 'DisplayName', ['2nd-Order RMSE=' num2str(rmse_2)]);
xlabel('Time (s)');
ylabel('Absolute Error (V)');
title('Prediction Error');
legend;
grid on;

% Pole-zero plot
subplot(2, 2, 3);
pzplot(sys_1);
hold on;
pzplot(sys_2);
title('Pole-Zero Locations');
grid on;

% Frequency response
subplot(2, 2, 4);
[mag1, phase1, w] = bode(sys_1);
[mag2, phase2, ~] = bode(sys_2);
loglog(w/(2*pi), squeeze(mag1), 'b-', 'LineWidth', 1.5, 'DisplayName', '1st-Order');
hold on;
loglog(w/(2*pi), squeeze(mag2), 'r-', 'LineWidth', 1.5, 'DisplayName', '2nd-Order');
xlabel('Frequency (Hz)');
ylabel('Magnitude');
```

```
title('Magnitude□Response');  
legend;  
grid on;  
  
sgtitle('Lab□14:□System□Identification□Analysis□|□Your□Name□|□Student□ID');
```

E Troubleshooting and Common Pitfalls

E.1 Data Acquisition Issues

- **No output from black box:** Check that the Arduino is connected to the TA and the black box is powered. Verify the PWM signal is reaching the RC filter input (use a multimeter).
- **Output is constant DC:** The black box may be in saturation. Reduce the amplitude of the test signal and try again.
- **Noisy measurements:** Add a low-pass filter to the Arduino ADC input (e.g., $1\text{ k}\Omega + 1\text{ }\mu\text{F}$). Increase the number of samples before averaging.

E.2 System Identification Issues

- **tfest() fails to converge:** Provide better initial guesses. Specify the pole and zero locations manually based on your step response observations.
- **First-order and second-order models fit equally well:** Use Occam's Razor: choose the simpler (first-order) model unless there is clear evidence of oscillation.
- **Simulink output does not match hardware:** Check that the transfer function is entered correctly. Verify the input signal $u(t)$ is identical to what was applied to the hardware.

E.3 Digital Twin Validation Issues

- **Large discrepancy between hardware and Simulink:** Your model may be missing high-frequency dynamics or nonlinearities. Increase the model order or investigate unmodeled inputs.
- **Small transient mismatch at the beginning:** The black box may have a small delay or dead-time element. Add a transport delay block in Simulink if necessary.

References and Inspiration

University Course Inspirations

This lab bridges fundamental Laplace domain analysis with practical reverse engineering and digital twin methodology:

- **MIT 6.003 (Signals, Systems, and Inference):** The Laplace Transform, pole-zero analysis, and the relationship between s -plane locations and time-domain response (natural frequency, damping ratio, settling time) are foundational to this lab. MIT's rigorous treatment of system properties via transfer functions enables the hand-derivation you perform in pre-lab work.
- **MIT 6.007 (Applied Electromagnetics):** The analog circuits underlying the mystery black box—RC networks, op-amp filtering, and dynamic servo systems—reflect MIT's emphasis on physical system realization. Understanding the circuit behavior from the Laplace domain is MIT's bridge between theory and hardware.
- **Georgia Tech ECE 2026 (Real-Time DSP):** Georgia Tech's expertise in **system identification and digital control** informs this lab's focus on measurement methodology, test signal design, and validation. The three application contexts (aerospace autopilot, industrial manufacturing, biomedical) reflect Georgia Tech's breadth in control systems across domains.
- **Rice University ELEC 241 (Real-Time DSP):** Rice's **emphasis on reverse engineering and practical model validation** is central to this lab. The lab teaches that you don't always have source code or documentation—you identify the system by measuring its input/output behavior and fitting a mathematical model. This is how embedded systems engineers maintain and upgrade legacy hardware.
- **Duke University ECE 280:** This final lab brings together all prior concepts—Fourier analysis (Labs 1–8), digital signal processing (Labs 9–10), and communication systems (Labs 11–12)—and applies them through the Laplace domain to a complete systems engineering problem: understanding unknown hardware and predicting its behavior.

Core References

- Franklin, G. F., Powell, J. D., & Emami-Naeini, A. (2015). *Feedback Control of Dynamic Systems* (7th ed.). Pearson.
 - **Chapters 2–4:** Laplace Transform properties, transfer functions, pole-zero analysis, and time-domain response prediction.
 - **Standard control systems textbook:** Used in undergraduate control courses worldwide; emphasis on hands-on system identification and design.
- Ljung, L. (1999). *System Identification: Theory for the User* (2nd ed.). Prentice Hall.
 - **Chapters 1–2, 6:** Rigorous treatment of how to identify linear systems from measurement data. Theory of model validation and error quantification.
 - **Industry standard:** The definitive reference for system identification methodology, used by engineers in aerospace, automotive, process control, and biomedical industries.
- Nise, N. S. (2019). *Control Systems Engineering* (8th ed.). Wiley.

- **Chapters 2–4:** Modeling systems with transfer functions, Laplace domain analysis, and prediction of transient response.
- **Pedagogical strength:** Clear derivations of pole-zero relationships to damping, settling time, and overshoot.
- The MathWorks. *System Identification Toolbox and Simulink Control Design Documentation*.
 - `tfest()`: Transfer function estimation from time-domain measurement data.
 - `idnlarx()`, `idpoly()`: More general system identification methods for nonlinear and higher-order systems.
 - **Digital twin workflow:** Using Simulink to simulate identified models against real hardware.
- Tao, F., Liu, W., & Nee, A. (2019). “Digital Twins: Past, Present, and Future.” *ASME Journal of Manufacturing Science and Engineering*, vol. 141, no. 6, 061014.
 - **Digital twin context:** Explains why identifying a system’s mathematical model is the first step in creating a digital twin—a virtual replica used for simulation, diagnostics, and design without risking the physical system.
 - **Real-world application:** Digital twins are foundational to predictive maintenance, failure analysis, and modernization of legacy equipment across aerospace, manufacturing, and biomedical industries.
- Blevins, R. D. (1979). *Formulas for Natural Frequency and Mode Shape*. Van Nostrand Reinhold.
 - **Mechanical systems context:** For the industrial legacy equipment application (Program 2), understanding natural frequencies and damping of hydraulic systems and mechanical components is essential to safe modernization.
- Ray, L. R., & Lee, H. (2015). *Robust State Estimation and Model Predictive Control for Process Systems*. Springer.
 - **Manufacturing application:** System identification applied to process control, with emphasis on handling model uncertainty and sensor noise.

Regulatory and Safety Standards

- FAA. *Advisory Circular 25.203: Stall Characteristics*.
 - **Aerospace context:** Aircraft autopilot systems must meet FAA requirements for stability and response characteristics. System ID is the first step in certifying digital autopilot replacements.
- FDA. *Guidance for Industry: Developing Biocompatible Medical Devices*.
 - **Biomedical context:** For clinical equipment analysis (Program 3), FDA requires that modernized systems be validated to meet the original equipment’s safety and performance characteristics. System ID provides the mathematical foundation for this validation.
- ISO 5349: *Mechanical Vibration and Shock*.

- **Industrial context:** Standard for characterizing vibration and dynamic response of industrial machinery. System ID helps factories predict machinery aging and plan maintenance.
- IEEE 1451: *Smart Sensor Interface Standard*.
- **Measurement standard:** How to interface sensors and data acquisition systems for reliable system identification experiments.

Why System Identification Is the Gateway to Modernization

In 2024, industries face a massive challenge: **aging, undocumented infrastructure**. Aircraft from the 1970s, factories with 30-year-old equipment, hospitals with vintage medical devices—all must continue operating safely despite lack of original documentation. System identification is the engineer’s tool to understand these unknowns.

By identifying a system’s transfer function from measurements, you unlock the ability to:

1. **Predict behavior:** Simulate the system under new conditions without physical testing (safe, cost-effective).
2. **Design replacements:** Create a modern digital controller that matches the original system’s behavior, preserving reliability while adding new features.
3. **Detect degradation:** Monitor whether the system’s transfer function is changing (early warning of component aging or failure).
4. **Meet regulatory requirements:** Provide mathematical evidence that modernized equipment will behave identically to the original in all conditions.

This lab teaches you to be the engineer who understands systems others consider *black boxes*—and that skill is invaluable in the real world.